

Verification across the cognition spectrum — modes, reach, and what catches a wrong correspondence

Status: design / planned (shelved). No experiments have been run. This is the fourth study of the first setting (transport-network service provisioning), shelved so that setting is worked thoroughly before the programme moves on. It reuses the framework, harness, invariants, and the instance oracle already built ([REPORT.md](#), [DESIGN_instance_disambiguation.md](#)); read the MAGIC paper's verification treatment first, which this operationalizes.

Summary

The first three studies measure *reconciliation* — do the agents propose the right correspondences, and what does the reference and the lift contribute. The MAGIC paper is equally emphatic about the step that makes reconciliation trustworthy: **verification**, which it treats as intrinsic, so that a correct translator is produced *by construction* rather than checked after the fact. This study makes the verification phase itself the object of demonstration and measurement, on the same first case, rounding the setting out.

The paper is precise about what verification is and is not. It is **not** round-trip equality: a forward-and-reverse translator pair built from the *same* mistaken correspondence passes a byte round-trip and is still wrong. Round-trip fidelity means agreement on **semantic invariants** — endpoint identity, connectivity, capacity, layer relationships, switching constraints, multiplexing structure — not byte equality. A proposed correspondence is **exercised by a virtual operation on the knowledge graph**: provision a service through it and confirm the resulting objects *in place*, independently of the reverse translator. Where the two forms share nothing (cross-domain) or the mapping is a refinement rather than an equivalence (intent), verification shifts to **satisfaction** — the realisation meets every stated bound — and to a **worked provisioning read-back**. And the *reach* of verification tracks the cognition spectrum: self-certifying between two live agents, evidence-and-round-trip on the invariants with one side inert, external adjudication or a downstream test when neither side can confirm.

The study asks three questions of the verifier itself, not the matcher: does it **pass what is correct and fail what is wrong** (a verifier has its own precision and recall); how far does its **reach** extend at each placement and by which mode; and what does it **cost**. The sharpest single demonstration is the one the paper names: construct a wrong correspondence whose forward-and-reverse pair passes a naive byte round-trip, and show the **invariant check and the virtual operation catch it** where round-trip equality does not.

1. Motivation and relation to the earlier studies

The main study's verify-and-repair step ([REPORT.md](#) §on verification) already separated the reference's *effort* role from its *correctness* role, and showed verification driving surviving false cognates toward zero. That was verification used as a *means* to a cleaner reconciliation. Here verification is the *object*: we vary the verification mode, the correspondence type, and the placement, and read the verifier's own quality and reach. The instance study contributes the mechanism — the provision-and-read-back oracle against invariants is exactly a verification act — so this study largely recomposes pieces already built, plus a seeded set of *wrong* correspondences to be caught.

2. The verification modes to compare

The paper uses several; naming them as discrete, comparable modes is half the contribution:

- **Byte round-trip** — translate $A \rightarrow B \rightarrow A$ and compare for equality. The naive baseline, and the one the paper explicitly warns is insufficient. Included precisely to be beaten.
- **Invariant round-trip** — translate and compare on the semantic invariants, not bytes. The paper's round-trip-fidelity-as-invariant-agreement.
- **Virtual operation on the graph** — exercise the correspondence by a provision-and-read-back and confirm the resulting objects in place against the invariants, independently of any reverse translator. The correctness-by-construction step; the instance oracle already implements it.
- **Satisfaction** — for a refinement (a bound met by a discrete realisation, as in the intent setting), verify that the realisation meets every stated bound; there is no round-trip because the mapping is lossy by design.
- **External adjudication** — the residual channel: a downstream test or a human sign-off, used where no live side can confirm.

3. The task

Given a set of **proposed correspondences** — the correct ones from the case gold, plus a seeded set of **wrong** ones (false cognates and subtler invariant-violating pairs) — a verifier must, for each, return **pass** or **fail** with the invariant(s) it turned on, under a given mode and placement. Scored against a **verification gold**: correct correspondences must pass, wrong ones must fail, and a correspondence the mode cannot decide at that placement must be **referred onward** (counted as reach, not as error).

4. Why the cognition spectrum matters here

Verification reach is gated by the same live-cognition access the instance study measures. A virtual operation needs a live side to provision on; an invariant round-trip needs enough structure to translate and compare; byte round-trip needs only the two static forms but proves the least. So:

- **both_cognitive** — the full mode set is available; verification is self-certifying and reach approaches complete.
- **one_inert** — the live side can be exercised; the inert side supports invariant round-trip and comparison but not a fresh provision. Reach is partial and mode-dependent.
- **both_inert** — no live operation; only static invariant comparison and, failing that, external adjudication. Reach is smallest; the residual is largest and most human.

The prediction is a **reach curve**: the fraction of correspondences a mode can decide falls as cognition recedes, steepest for the virtual operation (which needs a live side) and flattest for static invariant comparison.

5. Case construction

Reuses the first setting, adding only the seeded wrong correspondences and the verification gold; no new network:

- **Correct correspondences** — from `config_big_hard` (schema level) and `instance_hard` (instance level), which already carry invariant signatures.
- **Wrong correspondences to catch** — the planted false cognates (link vs trail termination; signal-grade vs service-grade; the two `svc-100` services), plus a few **invariant-violating** pairs constructed to pass a byte round-trip yet violate an invariant (e.g. a correspondence that preserves labels but swaps a capacity or an endpoint), which is the demonstration the paper calls for.
- **Refinement correspondences** — a small set from the intent-style mapping (a bound met by a discrete realisation) to exercise verification-by-satisfaction, distinct from equivalence.
- **Verification gold** — for each proposed correspondence: the expected verdict, the invariant that should catch a wrong one, and the modes/placements at which it is decidable. Derived and validated, never hand-scored.

6. Metrics

The verifier's own quality and reach, correctness-first:

- **Verification precision / recall** — of the correspondences it passes, the fraction truly correct; of the wrong ones, the fraction it fails (its catch rate). A verifier that passes everything has high reach but no discrimination; both are reported.
- **Catch rate by invariant** — which invariant turns on for each wrong correspondence (per-trap attribution, as in the reconciliation studies), so the mechanism is visible.
- **Reach** — the fraction of correspondences the mode can decide at each placement, and the residual referred onward (broken out by cause), giving the reach curve of §4.
- **Byte-round-trip false-pass rate** — the headline contrast: correspondences that pass a byte round-trip but are caught by the invariant check or the virtual operation.
- **Effort** — reasoning tokens; for the virtual-operation mode, the provision/interrogation counts already instrumented in the instance oracle.

7. Experimental design

A **mode × correspondence-type × placement × model** cross over the seeded verification set, with a validated gold, on the propose→verify shape already in the harness.

- **Modes**: byte round-trip, invariant round-trip, virtual operation, satisfaction, external (the last scored as "referred", not run).

- **Correspondence types:** equivalence (schema and instance), refinement.
- **Placements:** both_cognitive, one_inert, both_inert.
- **Models:** the strength ladder, for the capability interaction (does a weaker verifier miss invariant violations a stronger one catches?).
- **Trials:** ≥ 6 , checkpointed and en-route captured, as standing practice.

8. Pre-registered predictions

1. **Byte round-trip is beaten.** A constructed forward-and-reverse pair on a wrong correspondence passes byte round-trip and is caught by the invariant check and the virtual operation — the paper's claim, shown as a measured false-pass rate.
2. **The virtual operation has the highest catch rate and the steepest reach curve.** It catches the most wrong correspondences where a live side is available, and loses the most reach as cognition recedes.
3. **Reach falls with cognition; the mode you can still run changes.** both_cognitive self-certifies; both_inert falls back to static invariant comparison and external adjudication, with the largest referred residual.
4. **Capability-dependent verification.** Weaker models miss subtler invariant violations (a swapped capacity behind matching labels), so verification precision/recall degrades down the ladder — the verification analogue of the reconciliation capability gradient.
5. **Verification is where the reference's correctness role concentrates.** An invariant reference gives an inert side something to verify against, extending reach at one_inert beyond what static structure alone allows.

9. What it reuses, and what must be built

Reused as-is: the invariants, the instance **virtual-operation oracle** (provision-and-read- back against invariants), the cognition-spectrum treatment, the model ladder, the correctness-first discipline, checkpoint/resume, and the schema verify-and-repair scaffolding.

To build:

- a **verifier task** that takes proposed correspondences and returns per-correspondence verdicts with the invariant turned on, parameterized by **mode**;
- the **byte round-trip** and **invariant round-trip** modes (translate/compare), alongside the existing virtual-operation mode, and a **satisfaction** check for refinements;
- a **seeded verification set + gold**: the planted false cognates, a few constructed invariant-violating-but-byte-clean pairs, and the refinement correspondences, with the expected verdict and catching invariant per item;
- a driver, `verify_study.py`, structured like the others.

10. Open design questions

- **Constructing the byte-clean-but-wrong pair.** It must genuinely pass a byte round-trip yet violate an invariant; the cleanest is a correspondence that preserves the serialised form while swapping a capacity or endpoint the invariant pins. Worth getting exactly right, since it is the study's sharpest demonstration.
- **Satisfaction needs the intent form.** The refinement correspondences require the intent-style bounds-and-realisation pair; this study either reuses a small intent fragment or defers satisfaction to the intent setting (setting two) and scopes here to equivalence.
- **Verifier independence.** The paper stresses the virtual operation confirms *independently of the reverse translator*; the harness must keep the verifier from consulting the mapping it is checking, so a pass reflects the graph, not the translator's self-consistency.